

UNIT 4 · WEEKS 5–6

# Processes and memory

📅 Weeks 5–6 · dates to be confirmed    <🔗> Lab TD3 (solved + AI challenge)    ~ 7 animations

MANUAL M4 · THEORY OF THE UNIT

## Processes and memory

Core reading for unit 4 ( $\approx 2$  h, two weeks). Every section ends with a **Practice this** → link.  
The outputs are real: TD3 programs run on the course machine (Ubuntu 26.04, Linux 7.0, x86-64).

### CONTENTS

- |    |                                                        |    |                                                             |
|----|--------------------------------------------------------|----|-------------------------------------------------------------|
| §1 | The process and its address space                      | §4 | <code>fork</code> , copy-on-write and buffers               |
| §2 | Virtual memory: pages, faults and replacement          | §5 | <code>exec</code> , <code>exit</code> and <code>wait</code> |
| §3 | The process for the kernel: PCB, states and scheduling | §6 | The first process, sessions and services                    |

### §1 The process and its address space

A **process** is a running program together with its context: memory, registers, open descriptors, credentials. Every process has a **private address space**: the addresses it uses are *virtual*, and the same address in two processes points to different physical memory. On x86-64 pointers have 64 bits but 48 are used: 256 TiB, the lower half for the process and the upper half for the kernel.

The space is organized in **zones**: the *code* (text, read-only and executable), the *data* (initialized globals) and the *bss* (uninitialized globals), the *heap* (small `malloc`s), the *mmap* regions (libraries and large `malloc`s) and the *stack* (local variables, growing downwards). TD3's `zones` program prints an address of each zone and the `/proc/self/maps` region that contains it:

### Real output

```
$ ./zones
zone                address    perm  region
code   (main)        0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data   (initialized)   0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss    (global_array)  0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap   (malloc 10 B)    0x5f938dc96010  rw-p  [heap]
heap   (end: sbrk(0))  0x5f938dcb6fff  rw-p  [heap]
mmap   (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc   (printf)        0x71b345064080  r-xp  /usr/lib/x86_64-linux-gnu/libc.so.6
stack  (local)           0x7fff6c1e68b4  rw-p  [stack]
```

If you run it several times, the addresses change: the kernel randomizes the base of each region (**ASLR**, address space layout randomization) so that an attacker cannot predict where the code or the stack is. glibc serves `malloc`s of 128 KiB or more with a `mmap` of their own, outside the heap, which is returned to the system as soon as it is freed.

**Practice this** → TD3 · 5 ( zones )

## §2 Virtual memory: pages, faults and replacement

Memory is divided into **pages** (4 KiB on x86-64; also huge pages of 2 MiB and 1 GiB) and physical memory into **frames** of the same size. A per-process **page table** says in which frame each page is and with what permissions (read, write, execute, user). On x86-64 the table has 4 levels: a virtual address is split into four 9-bit indices and a 12-bit offset. The hardware —the **MMU**— translates on every access, and the **TLB** caches recent translations so as not to walk the table every time.

If the table entry is not present, the MMU raises a **page fault** and jumps into the kernel. If the address belongs to a valid region, the kernel allocates a frame, zeroes it (or reads it from disk) and retries the instruction: that is **lazy allocation**. If it belongs to no region, or the access violates the permissions (writing to code, executing on the stack), it sends `SIGSEGV`. Measured with `lazy`:

### Real output

```
$ ./lazy 1024
start                VmRSS =      1756 KiB    page faults so far = 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc          VmRSS =      1952 KiB    page faults so far = 8
after touching half of the pages  VmRSS =    526300 KiB    page faults so far = 131081
pages touched: 131072 (4096 bytes each)  page faults: 131073    1968 ns per fault
after free            VmRSS =      2012 KiB    page faults so far = 131081
```

Linux is also *optimistic* (overcommit): it promises more memory than it has, because hardly anyone uses everything they reserve. If it really runs out, the **OOM killer** terminates processes.

When there are no free frames left, the kernel picks a **victim** page and sends it to disk (*swap*) or compresses it in RAM (*zswap*, *zram*). The ideal algorithm (OPT) would evict the page used latest; real ones approximate LRU with the accessed bit set by the MMU (the clock algorithm; in Linux, active/inactive lists and MGLRU). With FIFO, **Belady's anomaly** can happen: more frames, more faults. If the set of pages in use does not fit in memory, the system spends more time paging than running: **thrashing**.

Practice this → A4.1 virtual memory · A4.5 page replacement

### §3 The process for the kernel: PCB, states and scheduling

The kernel describes every process with a **control block** (PCB; in Linux, `struct task_struct`): PID and PPID, owner and permissions, state, saved registers, priority, CPU times, descriptor table, limits (`getrlimit`), signals and page table. It can be inspected in `/proc/PID/`, which is what `ps` and `top` read.

A process goes through several **states**: *ready* (waiting for the CPU) and *running* —both `R` in `ps` —, *blocked* waiting for I/O (`S`, or `D` if it cannot be interrupted), *stopped* by `SIGSTOP` / `SIGTSTP` (`T`) and *zombie* (`Z`) between its end and its parent's `wait`.

Handing the CPU from one process to another is a **context switch**: saving one's registers, loading the other's and switching page tables. The **scheduler** decides whose turn it is. The classic algorithms: FCFS (arrival order; suffers the convoy effect), SJF and SRTF (shortest first; they minimize mean waiting but require estimating lengths), Round Robin (turns of one *quantum*; good response, more context switches) and priorities (with a risk of starvation, fixed with aging). Since version 6.6 (2023) Linux uses **EEVDF**, which shares the CPU by virtual time weighted by priority (`nice`) and serves the earliest deadline first.

Practice this → A4.3 states · A4.4 scheduling

### §4 `fork`, copy-on-write and buffers

In Unix a new process is born as a **clone** of another: `fork()` returns 0 in the child, the child's PID in the parent and -1 on failure. The child gets a copy of everything: memory, descriptors (pointing to the same open file table entries, unit 3), signal handlers and mask —but not pending signals—. Copying all the memory would be very expensive, so Linux uses **copy-on-write**: it copies only the page tables, marks everything read-only and copies each page only when one of the two writes to it:

### Real output

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096 bytes)
fork took 6.70 ms (only the page tables are copied)
child reads 65536 pages: 0 page faults (shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults (one copy per page), 2753 ns per copy
parent: the child exited with status 0; my copy still says 'x'
```

The copy also includes whatever the C library has pending. `printf` does not call `write`: it accumulates in a **buffer** that is flushed per line (stdout to a terminal), when it fills up (stdout to a pipe or a file), with `fflush` or at `exit`; stderr is unbuffered and `_exit` flushes nothing. If some text was still in the buffer at `fork`, both parent and child write it: it shows up twice.

### Real output

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

**Practice this → A4.2 copy-on-write · A4.6 buffers · TD3 · 2 and 3**

## §5 `exec`, `exit` and `wait`

`execve(path, argv, envp)` replaces the process's program: it loads new code and data, creates a new heap and stack and jumps to the entry point. It keeps the PID, the PPID and the open descriptors (except those marked `O_CLOEXEC`); with the SUID bit, the effective UID becomes the file owner's. If it works, it does not return. The library variants (`execl`, `execv`, `execvp` ...) change how arguments are passed and whether the `PATH` is searched. Between `fork` and `exec` the child can rearrange its descriptors: that is how the shell implements `>`, `<` and pipes (TD3 · 4's `launch`, and unit 5).

A process ends with `exit(status)` —which calls the `atexit` functions, flushes the stdio buffers and calls the `_exit` system call— or by a signal. The kernel closes its files and frees its memory, hands its living children over to `init` or the nearest *subreaper* and notifies the parent with `SIGCHLD`. The PCB stays as a **zombie** until the parent calls `wait` or `waitpid`, which returns the packed status; the macros decode it: `WIFEXITED` / `WEXITSTATUS`, `WIFSIGNALED` / `WTERMSIG`, `WIFSTOPPED` and `WIFCONTINUED` (with the `WUNTRACED` and `WCONTINUED` options; `WNOHANG` does not block).

### Real output

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has called wait yet:
    PID    PPID STAT COMMAND
    356667 356666 Z    lifecycle
waitpid → WIFEXITED = 1, WEXITSTATUS = 7; now ps finds nothing:
    PID    PPID STAT COMMAND
2) orphan: my PID is 356666
   grandchild 356679: parent was 9771, now it is 9771 (systemd)
```

### Real output

```
$ ./launch out.txt ./segfault
launch: ./segfault killed by signal 11 (SIGSEGV), core dumped
[exit status 139]
```

**Practice this → A4.7 fork, exec and wait · TD3 · 4**

## §6 The first process, sessions and services

If every process is born from a `fork`, someone has to be the first. **Process 0** is built by the kernel at boot (one per CPU, in kernel mode): it initializes virtual memory, creates the kernel threads (`kthreadd`, PID 2) and **process 1**, which switches to user mode and runs `/sbin/init` —today **systemd** in almost every distribution—, starts the services and adopts the orphans. On the course machine, `ps -p 1 -o comm=` answers `systemd` (TD0).

A **session** groups processes, typically those of a terminal, with a leader; if the leader ends, the others get `SIGHUP`. `setsid()` creates a new session without a controlling terminal. A classic **daemon** detached from the terminal with `fork`, the parent exiting and the child calling `setsid`. Today, on Linux, services are launched by `systemd`: the program runs in the foreground and `systemd` takes care of the session, the logs (`journalctl`), restarts and resource limits (`cgroups`, unit 9).